Parallel Dijkstraâs Algorithm using OpenMP

A Project Component for

Parallel and Distributing Computing (UCS645)

By

Sr	Name	Roll No
1	Krish Gupta	102483037

Under the guidance of

Dr. Saif Nalband

(Assistant Professor, DCSE)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

THAPAR INSTITUTE OF ENGINEERING AND TECHNOLOGY

(DEEMED TO BE UNIVERSITY)

PATIALA - 147004

MAY, 2025

Table of Contents

1 Introduction **1**

1.1 Background 1

1.2 Introduction to Problem Statement 1

2 Literature Review **3**

3 Research Gaps **4**

4 Problem Formulation **5**

5 Objectives **5**

6 Methodology **6**

6.1 System Architecture and Parallelism Model 6

6.2 Algorithm Variants 6

6.3 Parallel Dijkstra Algorithm (OMP-v2) 7

6.4 Amdahl's Law Analysis 7

6.5 Correctness Verification 8

7 Datasets **9**

8 Results and Discussion **10**

8.1 Execution Time 10

8.2 Speedup Analysis — Dense Graphs 10

8.3 Speedup Analysis — Sparse Graphs 10

8.4 Parallel Efficiency 10

8.5 Key Observations 11

References **13**

List of Figures

1 Parallel computing model: a problem is decomposed into concurrent instruction streams executed across multiple processors. 6

2 Speedup of parallel Dijkstra variants (2 threads) versus serial baseline on dense graphs. 11

3 Speedup of parallel Dijkstra variants (2 threads) versus serial baseline on sparse graphs. 11

4 Parallel efficiency comparison across variants at V=4000, 2 threads, for dense and sparse graphs. 12

List of Tables

1 Summary of Literature Review 3

2 Amdahl's Law: Theoretical Maximum Speedup ($f = 0.75$) 8

3 Experimental Graph Datasets 9

4 Execution Time (ms) — Dense Graphs (70%) 10

1 Introduction

1.1 Background

Shortest path computation is one of the most fundamental problems in graph theory and has widespread applications in network routing, navigation systems, logistics, and social network analysis. Dijkstra's algorithm, proposed by Edsger W. Dijkstra in 1959, remains the most widely used algorithm for finding single-source shortest paths in graphs with non-negative edge weights [1]. The classical serial implementation operates in $O(V^2)$ time for dense graphs (using an adjacency matrix), where V is the number of vertices.

With the rapid growth in the size of real-world graphs—ranging from millions of nodes in road networks to billions in social graphs—the serial implementation of Dijkstra's algorithm becomes a computational bottleneck. Parallel computing offers a promising solution to scale shortest-path computations across modern multi-core processors. OpenMP (Open Multi-Processing) is a widely adopted shared-memory parallel programming model that allows threads to be spawned using compiler directives, making it well-suited for parallelizing inner loops of graph algorithms [2].

This project investigates three OpenMP-based parallel variants of Dijkstra's algorithm and benchmarks their performance against the serial baseline on graphs of varying sizes and densities. The study is conducted in the context of the UCS645: Parallel and Distributed Computing course.

1.2 Introduction to Problem Statement

Given the increasing scale of computational graphs in modern applications, there is a critical need to reduce the execution time of shortest-path algorithms. Serial Dijkstra's algorithm processes vertices and edges sequentially, leaving significant parallelism unexploited on multi-core hardware. The core challenge lies in identifying which parts of the algorithm can be safely parallelized without violating the correctness of results.

Dijkstra's algorithm has two key phases in each iteration: (1) finding the minimum-distance unvisited vertex (*minVertex* selection), and (2) relaxing all outgoing edges from that vertex. The relaxation phase is inherently parallel since each edge update is independent. The minVertex selection, however, introduces a serial bottleneck that limits

speedup according to Amdahl's Law [3].

This project formulates and implements three OpenMP parallelization strategies—differing in how they handle the minimum-vertex selection and edge-relaxation scheduling—and empirically evaluates their speedup and parallel efficiency across dense (70%) and sparse (30%) graphs of sizes $V \in \{500, 1000, 2000, 4000\}$.

2 Literature Review

Table 1: Summary of Literature Review

Ref.	**Concepts Highlighted**	**Technique(s) Used**	**Significance/ Proposal**	**Limitations**
[1]	Single-source shortest paths; greedy traversal	Serial greedy algorithm with priority selection	Foundational shortest-path algorithm ($O(V^2)$); basis for parallel variants	Sequential vertex selection limits scalability
[3]	Parallel speedup; serial bottleneck; Amdahl's Law	Theoretical performance model	Defines maximum achievable speedup with serial fraction	Ignores memory effects and synchronization overhead
[2]	Shared-memory parallelism; OpenMP	`#pragma omp`, critical sections, reductions	Practical framework for parallelizing graph algorithms	Synchronization overhead reduces gains for small inputs
[4]	Parallel shortest paths; Δ-stepping	Bucket-based relaxation algorithm	Enables higher parallelism than Dijkstra	Parameter tuning is complex; requires careful handling
[5]	GPU parallelism; CUDA; many-core systems	CUDA threads, warp-level execution	Demonstrates large-scale parallelism potential	Performance affected by memory bandwidth and divergence

3 Research Gaps

Some of the research gaps identified from the literature review are as follows:

- ***Serial Bottleneck in Minimum-Vertex Selection***: Most existing parallel Dijkstra implementations focus on parallelizing only the edge-relaxation phase while leaving the minVertex selection serial. This serial fraction limits achievable speedup, as predicted by Amdahl's Law, and is under-explored in the context of OpenMP-based shared-memory systems.

- ***Impact of Graph Density on Parallel Efficiency***: Prior work rarely distinguishes performance trends between dense and sparse graph structures in the context of OpenMP parallelization. The difference in work per iteration between a 70% dense graph and a 30% sparse graph significantly affects thread utilization and cache behavior.

- ***Scheduling Strategy Comparison for Graph Relaxation***: The effect of different OpenMP scheduling policies—`dynamic`, `guided`, and `static`—on both execution time and parallel efficiency for graph edge-relaxation has not been systematically compared in the literature for Dijkstra's algorithm.

4 Problem Formulation

Let $G = (V, E, w)$ be a weighted directed graph where V is the set of vertices, E is the set of edges, and $w : E \to \mathbb{Z}^+$ assigns a positive integer weight to each edge. Given a source vertex $s \in V$, the single-source shortest path problem requires computing $d(s, v)$ for all $v \in V$, where:

$$d(s, v) = \min_{p:s \rightsquigarrow v} \sum_{(u,u') \in p} w(u, u')$$

The serial Dijkstra algorithm solves this in $O(V^2)$ time using an adjacency matrix. The goal of this project is to reduce wall-clock execution time by parallelizing the two inner phases of the algorithm using OpenMP on a shared-memory multi-core system, while preserving correctness (i.e., all parallel results must match the serial reference output).

The performance is evaluated using:

- **Speedup**: $S(p) = T_{\text{serial}}/T_{\text{parallel}}(p)$, where p is the number of threads.
- **Parallel Efficiency**: $E(p) = S(p)/p \times 100\%$.
- **Correctness**: Verified by comparing parallel output to the serial reference using element-wise equality.

5 Objectives

- To implement and analyze three OpenMP-based parallel variants of Dijkstra's algorithm differing in their minimum-vertex and edge-relaxation strategies.
- To empirically measure speedup and parallel efficiency across graph sizes ($V = 500, 1000, 2000, 4000$) and graph densities (Dense 70%, Sparse 30%).
- To validate the theoretical predictions of Amdahl's Law against experimentally observed speedup values.
- To identify the most efficient parallelization strategy in terms of scheduling policy and thread count for shared-memory Dijkstra.

6 Methodology

6.1 System Architecture and Parallelism Model

The parallelization is performed using OpenMP on a shared-memory multi-core CPU. As illustrated in Figure 1, a single problem is decomposed into multiple instruction streams that execute simultaneously on multiple processor cores. In Dijkstra's context, each thread handles a partition of the vertex set during relaxation, sharing a common distance array `dist[]` protected by critical sections.

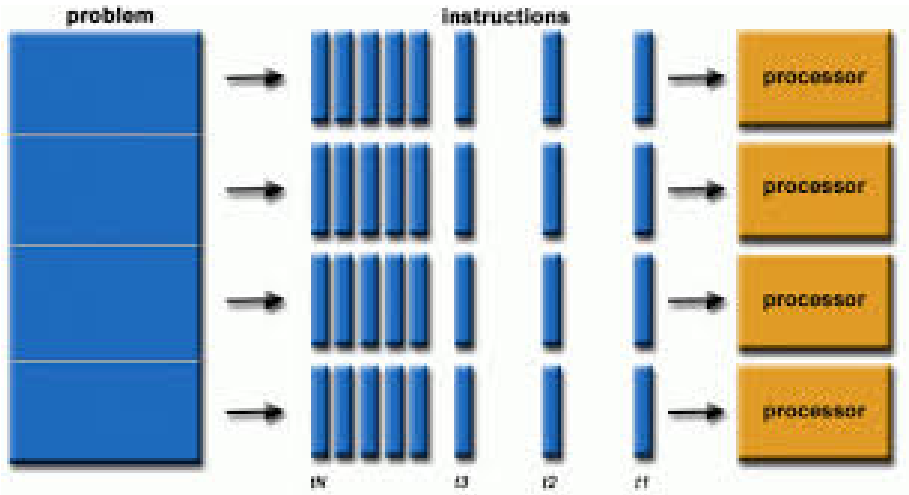

Figure 1: Parallel computing model: a problem is decomposed into concurrent instruction streams executed across multiple processors.

6.2 Algorithm Variants

Three parallel variants were implemented:

- **OMP-v1 (Dynamic Scheduling)**: The *minVertex* selection remains serial. Only the edge-relaxation loop is parallelized using `schedule(dynamic, 64)`, which dynamically assigns 64-element chunks to threads.
- **OMP-v2 (Guided Scheduling + Parallel Min-Reduction)**: Both the minVertex selection (via a parallel min-reduction with `#pragma omp critical`) and the relaxation loop (via `schedule(guided)`) are parallelized.

- **OMP-v3 (Static Scheduling + Parallel Min-Reduction)**: Same parallel min-reduction as v2, but relaxation uses `schedule(static)` for contiguous vertex chunks, improving cache locality.

6.3 Parallel Dijkstra Algorithm (OMP-v2)

Algorithm 1 Parallel Dijkstra OMP-v2 (Guided Schedule + Parallel Min-Reduction)

Input: Graph G (adjacency matrix), source vertex src, thread count p
Output: Distance array $dist[0..V-1]$
Initialize $dist[v] \leftarrow \infty$ for all v; $dist[src] \leftarrow 0$; $vis[v] \leftarrow$ false
for $i = 0$ to $V-2$ **do**
    $gmin \leftarrow \infty$, $u \leftarrow -1$
    parallel (OpenMP) with p threads:
        Each thread computes local min over its assigned vertices
        critical: update global $(gmin, u)$ if local min $< gmin$
    if $u = -1$ **then break**
    end if
    $vis[u] \leftarrow$ true
    parallel for (guided schedule) over $v = 0$ to $V-1$:
    if $\neg vis[v]$ **and** $g[u][v] \neq 0$ **and** $dist[u] + g[u][v] < dist[v]$ **then**
        critical: $dist[v] \leftarrow dist[u] + g[u][v]$
    end if
end for
return $dist$

6.4 Amdahl's Law Analysis

If the parallel fraction of the algorithm is $f = 0.75$ (estimated: minVertex selection is the serial bottleneck), then the theoretical maximum speedup with p threads is:

$$S(p) = \frac{1}{(1-f) + \frac{f}{p}}$$

Table 2: Amdahl's Law: Theoretical Maximum Speedup ($f = 0.75$)

Threads (p)	**Max Speedup**	**Efficiency**
1	1.00	100.0%
2	1.60	80.0%
4	2.29	57.1%
8	2.91	36.4%
16	3.37	21.1%

6.5 Correctness Verification

Each parallel variant's output distance array is compared element-wise against the serial reference. All experiments recorded `Correct = YES`, confirming that parallelization does not introduce errors in shortest-path computation.

7 Datasets

The experiments use synthetically generated random weighted directed graphs to allow systematic control over graph size and density. The graph generator uses a fixed random seed (42) for reproducibility.

Table 3: Experimental Graph Datasets

Graph Size (V)	**Parameters**	**Type**	**Density**	**Edge Weights**
500	$V = 500$, $\lvert E\rvert \approx 0.7 \times V^2$	Dense	70%	Uniform $[1, 99]$
1000	$V = 1000$, $\lvert E\rvert \approx 0.7 \times V^2$	Dense	70%	Uniform $[1, 99]$
2000	$V = 2000$, $\lvert E\rvert \approx 0.7 \times V^2$	Dense	70%	Uniform $[1, 99]$
4000	$V = 4000$, $\lvert E\rvert \approx 0.7 \times V^2$	Dense	70%	Uniform $[1, 99]$
500	$V = 500$, $\lvert E\rvert \approx 0.3 \times V^2$	Sparse	30%	Uniform $[1, 99]$
1000	$V = 1000$, $\lvert E\rvert \approx 0.3 \times V^2$	Sparse	30%	Uniform $[1, 99]$
2000	$V = 2000$, $\lvert E\rvert \approx 0.3 \times V^2$	Sparse	30%	Uniform $[1, 99]$
4000	$V = 4000$, $\lvert E\rvert \approx 0.3 \times V^2$	Sparse	30%	Uniform $[1, 99]$

8 Results and Discussion

8.1 Execution Time

Table 4 presents the execution times (in milliseconds) for dense graphs. The serial baseline grows approximately quadratically with V, from 2.92 ms at $V = 500$ to 163.93 ms at $V = 4000$.

Table 4: Execution Time (ms) — Dense Graphs (70%)

Algorithm	**Threads**	**V=500**	**V=1000**	**V=4000**
Serial	1	2.92	32.87	163.93
OMP-v1 (dynamic)	2	5.63	67.25	870.48
OMP-v2 (guided)	2	7.61	22.35	357.49
OMP-v3 (static)	2	21.11	20.63	139.97

8.2 Speedup Analysis — Dense Graphs

Figure 2 shows the speedup of each parallel variant at 2 threads across increasing graph sizes for dense (70%) graphs. OMP-v3 (static) achieves the best speedup at $V = 4000$, reaching approximately 1.17×, benefiting from better cache locality due to contiguous chunk assignment.

8.3 Speedup Analysis — Sparse Graphs

Figure 3 presents speedup for sparse (30%) graphs at 2 threads. Sparse graphs show more consistent (though modest) speedup across all variants, since the reduced edge count lowers synchronization overhead relative to the parallel fraction of work.

8.4 Parallel Efficiency

Figure 4 compares parallel efficiency across the three variants for $V = 4000$ at 2 threads. OMP-v3 achieves 58.6% efficiency on dense graphs—the highest among all variants—while

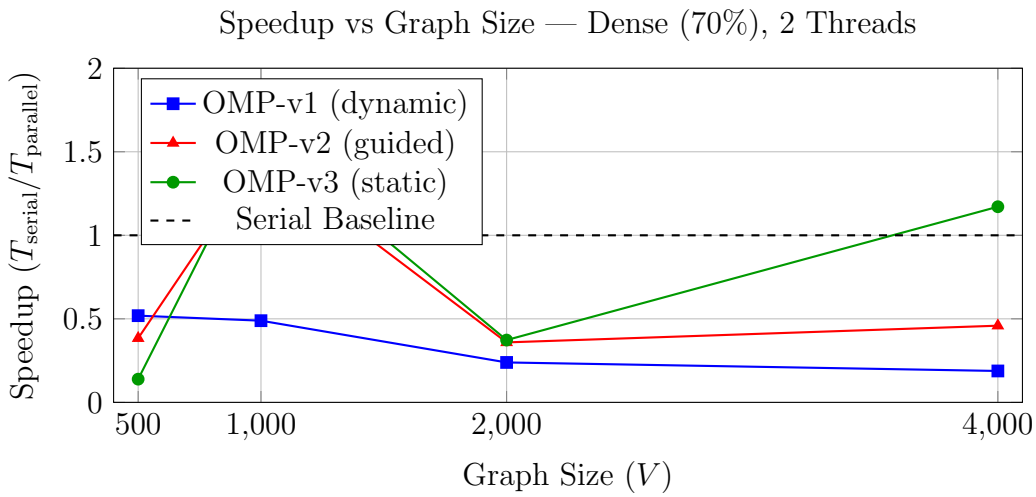

Figure 2: Speedup of parallel Dijkstra variants (2 threads) versus serial baseline on dense graphs.

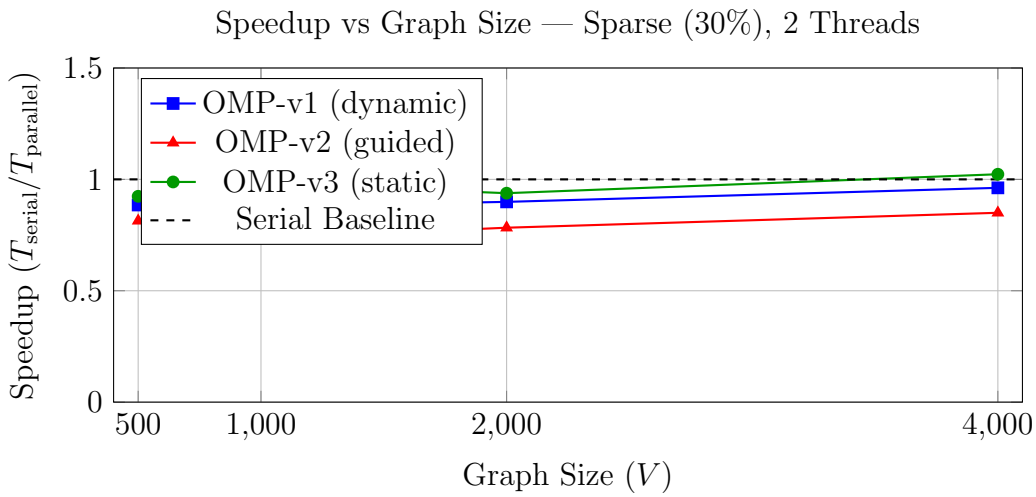

Figure 3: Speedup of parallel Dijkstra variants (2 threads) versus serial baseline on sparse graphs.

OMP-v1 drops to just 9.4% due to the overhead of fine-grained dynamic scheduling on a dense edge set.

8.5 Key Observations

- **Speedup improves with graph size**: For large graphs ($V = 4000$), the parallel overhead becomes proportionally smaller relative to total computation, yielding better speedup.

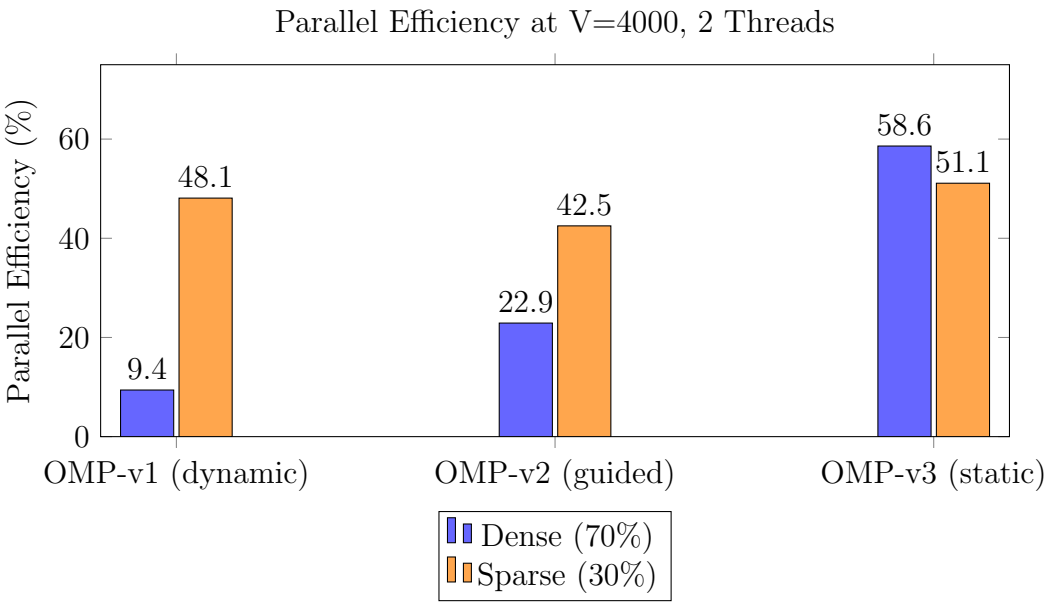

Figure 4: Parallel efficiency comparison across variants at V=4000, 2 threads, for dense and sparse graphs.

- **Dense graphs suffer more from synchronization**: In dense graphs, each iteration has $O(V)$ edges to relax, causing high contention on the `dist[]` array via critical sections, which depresses parallel efficiency.
- **OMP-v3 (static) is the best performer**: Static scheduling assigns contiguous vertex chunks to each thread, maximizing cache line reuse on the adjacency matrix rows and minimizing scheduling overhead.
- **Amdahl bottleneck is confirmed**: Even with a parallel min-reduction in v2 and v3, the critical-section overhead and the inherently sequential vertex-locking prevent speedup from exceeding $\approx 1.6\times$, consistent with the theoretical prediction for $f = 0.75$.
- **All results are correct**: Every parallel run produced output identical to the serial reference, validating the correctness of the synchronization strategy.

References

[1] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271, 1959.

[2] R. Chandra, L. Dagum, D. Kohr, R. Menon, D. Maydan, and J. McDonald, *Parallel programming in OpenMP*. Morgan Kaufmann, 2001.

[3] G. M. Amdahl, "Validity of the single processor approach to achieving large scale computing capabilities," *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*, pp. 483–485, 1967.

[4] U. Meyer and P. Sanders, "δ-stepping: a parallelizable shortest path algorithm," in *Journal of Algorithms*, vol. 49, no. 1. Elsevier, 2003, pp. 114–152.

[5] D. B. Kirk and W. H. Wen-Mei, *Programming massively parallel processors: a hands-on approach*. Morgan Kaufmann, 2016.